CastleInventoryAX is the master inventory of Castle construction assets.

It is where the ecosystem stores the building blocks, patterns, structures, definitions, and relationships needed to assemble applications consistently.

2. Why CastleInventoryAX Exists

As the Castle ecosystem grows, there may eventually be thousands of reusable application parts.

There may be:

- Thousands of Atomic Assets.
- Hundreds or thousands of Compounds.
- Hundreds of Composites.
- Many Castle Services.
- Many Castle Units.
- Many Blueprints.
- Many Castle Types.
- Many completed Castle structures.

Without CastleInventoryAX, the system would become difficult to manage. Developers and AI tools would not know what already exists. Similar assets would be duplicated. Patterns would become inconsistent. Reusable work would be lost inside individual applications.

CastleInventoryAX prevents that by creating a structured inventory.

Its goal is to make the ecosystem searchable, understandable, reusable, and governable.

If the ecosystem is like a construction company, CastleInventoryAX is not the customer list. It is not the billing system. It is not the lease agreement. It is the organized warehouse, catalog, plan library, material list, and assembly reference system for everything the company has designed or built.

3. The Correct Boundary of CastleInventoryAX

The boundary of CastleInventoryAX should be very clear.

CastleInventoryAX includes:

- Castle Types.
- Application Blueprints.
- Castle Units.
- Castle Services.
- Composites.
- Compounds.
- Atomic Assets.
- Asset relationships.
- Asset versions.
- Asset status.
- Approved patterns.
- Reusable structures.
- File structure templates.
- Initialization rules.
- Context inventory filters.
- Asset BOMs.
- Local modifications from a construction/inventory perspective.
- Build history of what was made.
- References showing where assets are used.

CastleInventoryAX does not include:

- Tenant ownership.
- Tenant billing.
- Customer subscription.
- Company account management.
- User access assignment.
- SSO assignment by customer.
- Payment information.
- Contracting information.
- Commercial deployment ownership.
- Who pays for the application.

- External account lifecycle management.

This does not mean those things are unimportant. It only means they belong somewhere else.

CastleInventoryAX should be cleanly defined as:

The inventory and relationship system for application structures and reusable construction assets.

4. Consistent Example Used Throughout

To keep the structure clear, this document will use one consistent example:

Strut Company Warehousing and Inventory Castle

This is an internal warehousing and inventory application structure created inside the Castle ecosystem. The example is used to explain how the Castle structure would be inventoried in CastleInventoryAX.

In this example:

Castle Name: `Strut Company Warehousing and Inventory Castle`

Castle Record ID: `CSTL-STRUT-WAREHOUSE-INVENTORY-V001`

Castle Type: `Internal Inventory Castle`

Blueprint: `Inventory Application - Internal Tenant`

Primary Castle Unit: `Warehousing and Inventory Castle Unit`

Castle Services:

- Auth Castle Service
- Permission Castle Service
- Audit Log Castle Service
- Inventory Castle Service

- Warehouse Location Castle Service
- Stock Adjustment Castle Service
- Reporting Castle Service

Composites:

- Inventory Table Composite
- Item Detail Page Composite
- Warehouse Location Selector Composite
- Quantity Adjustment Drawer Composite
- Inventory Dashboard Widget Composite

Compounds:

- Date Range Filter Compound
- Role Check Compound
- Quantity Validation Compound
- Pagination Compound
- Stock Status Filter Compound

Atomic Assets:

- `InventoryStatusEnum`
- `WarehouseLocationType`
- `formatQuantity()`
- `validatePositiveNumber()`
- `canAdjustInventory`
- `StockStatusBadge`
- `InventoryPermissionConstant`

This example does not require Tenant ID. Strut Company is used here as the name of the example Castle/application structure, not as a tenant-management record.

CastleInventoryAX is concerned with the fact that this Castle exists, what it is made from, what Blueprint initialized it, what reusable assets it uses, what local construction modifications it contains, and what parts could be reused later.

5. Castle

A **Castle** is the complete application or system structure.

In CastleInventoryAX, a Castle is inventoried as a finished or working application structure that was created from a Blueprint and assembled from smaller reusable parts.

For the example:

Castle: `Strut Company Warehousing and Inventory Castle`

This Castle represents the full warehousing and inventory application structure. It includes the front end, back end, app shell, pages, services, routes, permissions, components, patterns, and reusable assets used to create the application.

A Castle answers:

What complete application structure exists?

CastleInventoryAX should store a Castle record with information such as:

- Castle Record ID.
- Castle name.
- Castle Type.
- Blueprint used.
- Version.
- Status.
- Description.
- Included Castle Units.
- Included Castle Services.
- Included Composites.
- Included Compounds.
- Included Atomic Assets.
- Local modifications.
- Asset BOM.
- Build notes.
- Review status.
- Reuse recommendations.

For the example, CastleInventoryAX would store:

Castle Record ID: `CSTL-STRUT-WAREHOUSE-INVENTORY-V001`

Castle Name: `Strut Company Warehousing and Inventory Castle`

Castle Type: `Internal Inventory Castle`

Blueprint: `Inventory Application - Internal Tenant`

Status: Active / Built / In Review / Reusable, depending on lifecycle

Primary Purpose: Warehousing and inventory management structure

The Castle record does not need to know who pays for it, who owns it commercially, or who has access to it. It only needs to know what the Castle is and what it is made from.

6. Castle Type

A **Castle Type** defines what kind of application the Castle is.

It is a classification layer. It helps CastleInventoryAX group similar application structures together.

For the example:

Castle Type: `Internal Inventory Castle`

This means the Strut Company Warehousing and Inventory Castle belongs to the category of internal inventory applications.

A Castle Type answers:

What kind of application structure is this?

Castle Types may include:

- Internal Inventory Castle.
- Workflow Castle.
- Reporting Castle.
- Admin Console Castle.
- Vendor Portal Castle.
- Customer Portal Castle.
- Marketplace Castle.

- Document Management Castle.
- Compliance Castle.
- Communication Castle.
- Learning Castle.
- Scheduling Castle.

The Castle Type is important because it tells the system what general kind of assets, Blueprints, and patterns are likely relevant.

For example, an Internal Inventory Castle will likely need:

- Inventory pages.
- Location structures.
- Quantity adjustment patterns.
- Stock status logic.
- Inventory audit logs.
- Warehouse permissions.
- Inventory reporting.
- Item detail pages.

A Marketplace Castle would need very different things, such as product listings, vendor profiles, customer accounts, carts, checkout, and public browsing.

CastleInventoryAX should use Castle Type as one of the first filters for organizing and retrieving relevant assets.

7. Blueprint

A **Blueprint** is the approved starter structure used to initialize a Castle.

The Blueprint is not the finished application. It is the application starting system.

It answers:

How should this kind of Castle be structurally initialized?

For the example:

Blueprint: `Inventory Application - Internal Tenant`

This Blueprint would define the standard structure for creating an internal inventory application.

It may include:

- Front-end file structure.
- Back-end file structure.
- App shell.
- Navigation.
- Login structure.
- Base auth pattern.
- Admin area.
- User structure.
- Base permissions.
- Page placeholders.
- Layouts.
- Drawers.
- Menus.
- Shared components.
- Design system.
- API conventions.
- Logging standards.
- Error handling patterns.
- Context inventory filters.
- Initialization rules.

The Blueprint should produce a working but minimal application shell.

For the Strut Company Warehousing and Inventory Castle, the Blueprint might initialize:

Front end:

- Dashboard page.
- Inventory page.
- Item detail page.
- Warehouse locations page.
- Reports page.
- Admin page.
- Settings page.
- App shell.

- Sidebar navigation.
- Empty states.

Back end:

- Auth module.
- Users module.
- Roles module.
- Permissions module.
- Inventory module placeholder.
- Warehouse locations module placeholder.
- Audit log module.
- API conventions.
- Base services.
- Environment configuration.

The important distinction is:

The Blueprint creates the starting structure. It does not complete the final business logic.

It may create placeholder pages, base routing, standard folders, and approved patterns. The detailed features are built after the Blueprint is initialized.

CastleInventoryAX should store Blueprint records so the system knows which starter structures exist and which Castle Types they support.

8. Castle Unit

A **Castle Unit** is a major section, domain area, or organized part inside a Castle.

Castle Units divide the Castle into meaningful application areas.

For the example:

Primary Castle Unit: `Warehousing and Inventory Castle Unit`

The Strut Company Warehousing and Inventory Castle may include these Castle Units:

- Warehousing and Inventory Castle Unit.
- Admin Castle Unit.
- Reporting Castle Unit.
- User Management Castle Unit.

A Castle Unit answers:

What major area of the Castle does this part belong to?

The Warehousing and Inventory Castle Unit may include:

- Inventory item lists.
- Item detail views.
- Warehouse locations.
- Stock adjustments.
- Transfers.
- Reconciliation.
- Inventory-specific audit events.
- Inventory-specific reporting.
- Inventory permissions.

The Admin Castle Unit may include:

- User management.
- Role settings.
- System settings.
- Configuration screens.

The Reporting Castle Unit may include:

- Inventory dashboard.
- Stock reports.
- Monthly reconciliation reports.
- Export views.
- Saved report filters.

The User Management Castle Unit may include:

- User list.
- User roles.
- Permission assignments.

- Profile settings.

CastleInventoryAX should store Castle Units because they help organize assets at a higher level than individual services or components. They make the Castle easier to understand, reuse, and maintain.

9. Castle Service

A **Castle Service** is a governed functional capability inside a Castle or Castle Unit.

This term replaces the generic word “service” because it is more specific to the Castle ecosystem.

A Castle Service may include backend logic, API routes, database interactions, contracts, observability, configuration screens, admin controls, logs, health checks, validation rules, and frontend visibility where needed.

For the example, the Strut Company Warehousing and Inventory Castle may include:

- Auth Castle Service.
- Permission Castle Service.
- Audit Log Castle Service.
- Inventory Castle Service.
- Warehouse Location Castle Service.
- Stock Adjustment Castle Service.
- Reporting Castle Service.

A Castle Service answers:

What functional system capability exists inside this Castle?

For example, the **Inventory Castle Service** may include:

- Inventory API routes.
- Inventory service logic.
- Inventory database schema.
- Inventory validation rules.
- Inventory permissions.

- Inventory event logs.
- Inventory table connections.
- Inventory admin settings.
- Inventory health checks.

The **Warehouse Location Castle Service** may include:

- Location records.
- Warehouse area structure.
- Bin, shelf, zone, or room logic.
- Location selector components.
- Location validation.
- Location APIs.
- Location reports.

The **Stock Adjustment Castle Service** may include:

- Quantity adjustment rules.
- Adjustment reasons.
- Approval thresholds.
- Adjustment history.
- Adjustment audit events.
- Adjustment drawer UI.
- Adjustment permissions.

CastleInventoryAX should inventory Castle Services because they are major reusable functional systems. They are bigger than individual UI components or helper functions, but smaller than the full Castle.

10. Composite

A **Composite** is a larger reusable feature block.

It is usually visible or functional enough that a user, admin, or developer can recognize it as a feature block.

For the example, Composites may include:

- Inventory Table Composite.
- Item Detail Page Composite.
- Warehouse Location Selector Composite.
- Quantity Adjustment Drawer Composite.
- Inventory Dashboard Widget Composite.

A Composite answers:

What reusable feature block exists?

For example, the **Inventory Table Composite** may include:

- Data table layout.
- Search bar.
- Filter controls.
- Stock status badges.
- Pagination.
- Row actions.
- Empty state.
- Loading state.
- Permission-aware buttons.

The **Quantity Adjustment Drawer Composite** may include:

- Drawer layout.
- Quantity input.
- Reason selector.
- Approval notice.
- Submit button.
- Cancel button.
- Validation messages.
- Audit log trigger.

The Composite is important because it is often the level where business users and developers both understand the feature. A user may not know what Atomic Assets are inside the Inventory Table, but they can understand the Inventory Table as a feature block.

CastleInventoryAX should store Composite records with:

- Composite ID.
- Name.
- Description.
- Version.
- Status.
- Related Castle Type.
- Related Blueprint.
- Related Castle Services.
- Required Compounds.
- Required Atomic Assets.
- Usage references.
- Approval status.
- Reuse notes.

This allows the ecosystem to reuse Composites across multiple Castles.

11. Compound

A **Compound** is a smaller reusable assembly made from multiple Atomic Assets.

It is not as large as a Composite, but it is more than one tiny part. It usually represents grouped logic, validation, filtering, formatting, or behavior.

For the example, Compounds may include:

- Date Range Filter Compound.
- Role Check Compound.
- Quantity Validation Compound.
- Pagination Compound.
- Stock Status Filter Compound.

A Compound answers:

What reusable grouped logic or small assembly exists?

For example, the **Quantity Validation Compound** may include:

- Positive number validation.

- Decimal handling.
- Unit-of-measure validation.
- Minimum quantity rule.
- Error message helper.
- Quantity formatting helper.

The **Role Check Compound** may include:

- Permission constant.
- User role lookup.
- Role comparison helper.
- Access decision helper.
- Error handling for unauthorized actions.

Compounds are important because they keep small pieces of reusable logic organized. Without Compounds, the system may have thousands of Atomic Assets but no practical grouping layer between tiny pieces and larger feature blocks.

CastleInventoryAX should show which Atomic Assets belong to each Compound and which Composites or Castle Services use that Compound.

12. Atomic Asset

An **Atomic Asset** is the smallest reusable building block in the Castle ecosystem.

Atomic Assets may include:

- Helper functions.
- Validators.
- Constants.
- Types.
- Enums.
- Small UI elements.
- Formatting functions.
- Query helpers.
- Permission checks.
- Small reusable logic blocks.

For the example, Atomic Assets may include:

- `InventoryStatusEnum`
- `WarehouseLocationType`
- `formatQuantity()`
- `validatePositiveNumber()`
- `canAdjustInventory`
- `StockStatusBadge`
- `InventoryPermissionConstant`

An Atomic Asset answers:

What is the smallest reusable part available in the inventory?

Atomic Assets matter because they are the foundation of reuse.

If the ecosystem eventually contains 10,000 or more Atomic Assets, CastleInventoryAX becomes essential. The system must be able to filter, categorize, validate, and retrieve the right Atomic Assets without requiring AI or developers to search everything manually.

For example, when building or modifying the Strut Company Warehousing and Inventory Castle, the system should be able to retrieve inventory-related Atomic Assets first, such as:

- Inventory status enums.
- Warehouse location types.
- Quantity validators.
- Stock badge components.
- Inventory permission constants.

This keeps development consistent and prevents unnecessary duplication.

13. Local Modifications

Local Modifications are changes made to a specific Castle record that differ from the standard Blueprint, Castle Type, Castle Unit, Castle Service, Composite, Compound, or Atomic Asset configuration.

In CastleInventoryAX, Local Modifications are tracked from an inventory and construction perspective, not a tenant-management perspective.

For the example, the Strut Company Warehousing and Inventory Castle may include Local Modifications such as:

- Custom warehouse location naming.
- Custom role called `WarehouseSupervisor`.
- Quantity adjustment approval required above a defined threshold.
- Separate tracking for raw materials, work-in-progress, and finished goods.
- Custom inventory report for monthly warehouse reconciliation.

A Local Modification answers:

What was changed for this specific Castle structure?

This matters because CastleInventoryAX must distinguish between standard reusable assets and Castle-specific changes.

For example, if the Strut Company Warehousing and Inventory Castle adds a custom approval threshold for stock adjustments, that does not automatically mean every Internal Inventory Castle should use that same threshold.

CastleInventoryAX should record the change as a Local Modification unless the team later decides to promote it into a reusable Compound, Composite, Castle Service, or Blueprint update.

A Local Modification record may include:

- Modification ID.
- Castle Record ID.
- Modified asset or structure.
- Change description.
- Reason for change.
- Related Blueprint.
- Related Castle Service.
- Review status.
- Whether it should remain local.
- Whether it should be promoted into a reusable asset.
- Testing notes.
- Version notes.

This helps the ecosystem learn from local changes without accidentally turning every local decision into a global standard.

14. Asset BOM

The **Asset BOM**, or Asset Bill of Materials, is the full list of parts used to assemble a Castle.

In CastleInventoryAX, the Asset BOM is one of the most important outputs.

It answers:

What is this Castle made from?

For the Strut Company Warehousing and Inventory Castle, the Asset BOM would include:

- Castle Type.
- Blueprint.
- Castle Units.
- Castle Services.
- Composites.
- Compounds.
- Atomic Assets.
- Local Modifications.
- Versions.
- Statuses.
- Dependencies.
- Reuse notes.

A simplified Asset BOM could look like:

Castle: `Strut Company Warehousing and Inventory Castle`
Castle Record ID: `CSTL-STRUT-WAREHOUSE-INVENTORY-V001`
Castle Type: `Internal Inventory Castle`
Blueprint: `Inventory Application - Internal Tenant`
Castle Unit: `Warehousing and Inventory Castle Unit`

Castle Service: `Stock Adjustment Castle Service`

Composite: `Quantity Adjustment Drawer Composite`

Compound: `Quantity Validation Compound`

Atomic Assets: `validatePositiveNumber()`, `formatQuantity()`, `InventoryStatusEnum`

Local Modification: `WarehouseSupervisor approval required above defined threshold`

The Asset BOM allows the team to trace impact.

For example, if `validatePositiveNumber()` has a defect, CastleInventoryAX should show that it is used in:

- Quantity Validation Compound.
- Quantity Adjustment Drawer Composite.
- Stock Adjustment Castle Service.
- Strut Company Warehousing and Inventory Castle.

That means a small Atomic Asset can be traced all the way up to the Castle level.

This is one of the most important reasons CastleInventoryAX exists.

15. How CastleInventoryAX Supports AI Development

CastleInventoryAX is especially valuable because it gives AI clear, structured context.

Without CastleInventoryAX, AI may retrieve unrelated code, invent new patterns, duplicate existing logic, or build outside approved structures.

With CastleInventoryAX, AI can follow the correct assembly path.

The flow should be:

New Castle Request

↓

Select Castle Type

↓

Select Blueprint

↓

Load Blueprint Rules

↓

Filter CastleInventoryAX

↓

Retrieve Relevant Castle Units

↓

Retrieve Relevant Castle Services

↓

Retrieve Relevant Composites

↓

Retrieve Relevant Compounds

↓

Retrieve Relevant Atomic Assets

↓

Initialize Castle Structure

↓

Identify Gaps

↓

Generate Only Missing Pieces

↓

Record New Assets Back Into CastleInventoryAX

For the Strut Company Warehousing and Inventory Castle, AI should not search the entire ecosystem. It should first understand:

- This is an Internal Inventory Castle.
- It uses the Inventory Application - Internal Tenant Blueprint.
- It should prioritize inventory, warehousing, stock adjustment, reporting, audit log, and permission assets.

Then it should retrieve relevant assets such as:

- Inventory table patterns.
- Item detail page patterns.
- Quantity adjustment drawer.

- Warehouse location selector.
- Stock status badges.
- Inventory audit log patterns.
- Warehouse permission models.
- Inventory schema templates.

This reduces token waste, improves quality, and keeps AI inside approved application guardrails.

CastleInventoryAX therefore becomes the structured retrieval source for AI-assisted building.

16. What CastleInventoryAX Should Store

CastleInventoryAX should store records for each major inventory layer.

Castle Records

Castle records define completed or working application structures.

They should include:

- Castle Record ID.
- Castle name.
- Castle Type.
- Blueprint used.
- Version.
- Status.
- Purpose.
- Included Castle Units.
- Included Castle Services.
- Asset BOM.
- Local Modifications.
- Build notes.
- Review notes.

Castle Type Records

Castle Type records define application categories.

They should include:

- Castle Type ID.
- Name.
- Description.
- Common purpose.
- Typical use cases.
- Compatible Blueprints.
- Default Castle Units.
- Default Castle Services.
- Recommended asset filters.

Blueprint Records

Blueprint records define starter application structures.

They should include:

- Blueprint ID.
- Name.
- Category.
- Version.
- Status.
- Purpose.
- What it is for.
- What it is not for.
- Front-end structure.
- Back-end structure.
- Auth assumptions.
- User model assumptions.
- Navigation assumptions.
- Default pages.
- Default components.
- Context inventory filters.
- Initialization rules.

- Placeholder rules.
- Required review steps.

Castle Unit Records

Castle Unit records define major application areas.

They should include:

- Castle Unit ID.
- Name.
- Description.
- Related Castle Type.
- Related Blueprint.
- Included Castle Services.
- Included Composites.
- Permission scope.
- Domain notes.

Castle Service Records

Castle Service records define functional system capabilities.

They should include:

- Castle Service ID.
- Name.
- Capability.
- Related Castle Unit.
- Backend modules.
- API contracts.
- Database interactions.
- Frontend visibility.
- Admin controls.
- Observability.
- Logging.
- Health checks.
- Permission rules.
- Related Composites.

Composite Records

Composite records define larger reusable feature blocks.

They should include:

- Composite ID.
- Name.
- Description.
- Version.
- UI/backend scope.
- Related Castle Service.
- Required Compounds.
- Required Atomic Assets.
- Compatible Blueprints.
- Usage references.
- Status.

Compound Records

Compound records define grouped reusable logic.

They should include:

- Compound ID.
- Name.
- Description.
- Included Atomic Assets.
- Used by Composites.
- Used by Castle Services.
- Version.
- Status.
- Testing notes.

Atomic Asset Records

Atomic Asset records define the smallest reusable parts.

They should include:

- Atomic Asset ID.
- Name.
- Type.
- Description.
- Code location.
- Version.
- Status.
- Dependencies.
- Used by Compounds.
- Used by Composites.
- Used by Castle Services.
- Used by Castles.
- Validation notes.
- Approved pattern notes.

Local Modification Records

Local Modification records define Castle-specific changes.

They should include:

- Modification ID.
- Castle Record ID.
- Modified item.
- Description.
- Reason.
- Related asset.
- Review status.
- Promotion recommendation.
- Testing notes.

17. Relationship Structure Inside CastleInventoryAX

CastleInventoryAX is not just a flat list. It is a relationship system.

The basic relationship structure is:

Castle

↓

Castle Type

↓

Blueprint

↓

Castle Units

↓

Castle Services

↓

Composites

↓

Compounds

↓

Atomic Assets

↓

Local Modifications

Using the Strut Company Warehousing and Inventory Castle example:

`CSTL-STRUT-WAREHOUSE-INVENTORY-V001` is the Castle record.

It is classified as an `Internal Inventory Castle`.

It was initialized from the `Inventory Application - Internal Tenant` Blueprint.

That Blueprint created the starting structure for inventory, admin, reporting, and user management areas.

The `Warehousing and Inventory Castle Unit` contains the Inventory Castle Service, Warehouse Location Castle Service, and Stock Adjustment Castle Service.

The `Stock Adjustment Castle Service` uses the `Quantity Adjustment Drawer Composite`.

The `Quantity Adjustment Drawer Composite` uses the `Quantity Validation Compound` and `Role Check Compound`.

The `Quantity Validation Compound` uses Atomic Assets such as `validatePositiveNumber()`, `formatQuantity()`, and `InventoryStatusEnum`.

The Castle also includes a Local Modification requiring Warehouse Supervisor approval for quantity adjustments above a defined threshold.

This structure allows the team to understand what exists, how it is connected, and what would be impacted by a change.

18. What CastleInventoryAX Produces

CastleInventoryAX should produce clear outputs.

These outputs may include:

- Castle inventory list.
- Blueprint library.
- Castle Type library.
- Castle Unit library.
- Castle Service library.
- Composite library.
- Compound library.
- Atomic Asset library.
- Asset BOM for each Castle.
- Asset dependency map.
- Reuse reports.
- Deprecated asset reports.
- Duplicate asset detection.
- Local modification reports.
- AI retrieval filters.
- Build readiness reports.
- Approval status reports.

For the Strut Company Warehousing and Inventory Castle, CastleInventoryAX should be able to produce a report showing:

- What Castle was created.
- What type of Castle it is.
- What Blueprint initialized it.
- What Castle Units it contains.

- What Castle Services it uses.
- What Composites are included.
- What Compounds are included.
- What Atomic Assets are included.
- What local modifications were made.
- Which pieces are reusable.
- Which pieces are custom.
- Which pieces may need review.

20. Final Summary Definition

CastleInventoryAX is the structured inventory system for the Castle ecosystem's application assets, Blueprints, Castle Types, Castle Units, Castle Services, Composites, Compounds, Atomic Assets, Asset BOMs, and Local Modifications.

It is not a Tenant system.

It is not a billing system.

It is not an access control system.

It is not the place to define who owns or uses a deployed application.

Its purpose is to maintain a clear, searchable, reusable, and governed inventory of what has been created and what can be assembled.

Using the **Strut Company Warehousing and Inventory Castle** example:

The Castle is `CSTL-STRUT-WAREHOUSE-INVENTORY-V001`.

It is an `Internal Inventory Castle`.

It was initialized from the `Inventory Application - Internal Tenant` Blueprint.

It contains Castle Units such as Warehousing and Inventory, Admin, Reporting, and User Management.

Those Castle Units contain Castle Services such as Auth, Permissions, Audit Log, Inventory, Warehouse Location, Stock Adjustment, and Reporting.

Those Castle Services use Composites such as the Inventory Table, Item Detail Page, Warehouse Location Selector, Quantity Adjustment Drawer, and Inventory Dashboard Widget.

Those Composites use Compounds such as Quantity Validation, Role Check, Pagination, Date Range Filtering, and Stock Status Filtering.

Those Compounds use Atomic Assets such as validators, enums, constants, helper functions, permission checks, and small UI components.

Any Strut-specific construction changes are stored as Local Modifications.

CastleInventoryAX records and governs all of this so the ecosystem can know what exists, reuse what has already been made, reduce duplicated work, guide AI retrieval, trace asset dependencies, and assemble new Castles from approved and understandable parts.

I need you to clean up this document and information below. Remove reference to billing, and conversations of why something is not in this application. I do not need information not about CastleInventoryAX. I need a clear break down o the application and what it is. There can be a single summary of 2 sentences of what is out of scope only. Provide one clear Example of a Warehouse Inventory Application as the Use Case.